

Custom Implementation Guide

The following includes a list of functions that should be implemented in a working custom.cc implementation. This guide is not meant to give you the answer, but instead list functionality that you should consider when implementing the code in custom.cc. You may find an implementation that works that has many differences with this guide. **This guide is a very high-level overview that only provides some important functionality but does not include everything needed for a correct implementation.** Please consult the lecture slides and the “Project 2 FAQ” document for more information.

Constructor():

- *calculate the PHThistoryBits bits using the PredictorSize*
- *calculate the bBranchAddressBits*
- *calculate the BranchMask*
- *calculate the globalHistoryMask*
- *calculate the PHTThreshold*

uncondBranch():

- *modify the history using the thread.*
- *modify the prediction attribute and global history attribute of the history.*

squash():

- *modify the globalHistoryReg of the thread.*
- *delete the history.*

lookup():

- *define ‘n’ bits of global history and ‘m’ bit of branch address.*
- *calculate the PHTIdx using ‘m’ and ‘n’.*
- *modify the history using the thread to determine a prediction.*

btbUpdate():

- *modify the globalHistoryReg of the thread.*

update():

- modify the history.
- if squashed {do something}, otherwise {do something else}.
- *calculate PHTIdx using m and n .*
- *update PHTCounters accordingly to whether the branch is taken or not.*
- delete the history.

NOTE: You may implement other functions if needed to modify the global history based on thread and whether or not the branch was taken. You may also implement other parameters.